Lab 2 Reflection — Bridge Pattern

Riejed Aniko L. Macatula | BSIT 3A | Integrative Programming and Technologies

The smell

The without-bridge version splits into three separate classes: AttendanceJsonGzipReport, AttendanceJsonPlainReport, and AttendanceTextGzipReport. Each one repeats the same file_get_contents call to fetch time-log data, then handles compression slightly differently. Two dimensions are tangled into one inheritance chain: how the report is formatted, and whether the output gets gzipped. Two formats times two compression choices already forces four classes, each re-implementing the fetch step from scratch. Add a third format or a second compression scheme and the class count multiplies rather than adds, since every combination needs its own subclass.

The fix

Bridge separates the two dimensions. TimesFormatter is the abstraction: it holds a DataSource and a Compressor, and its generate() method fetches records, converts them to text through the abstract toText(), then hands the result to the compressor. DataSource and Compressor are separate implementor hierarchies (ApiDataSource/FileDataSource on one side, GzipCompressor/NoneCompressor on the other), and neither knows the other exists. AttendanceTimesFormatter and TimesheetJsonFormatter differ only in toText(); everything else, including the ability to swap either implementor, is inherited.

I implemented setCompressor() so the abstraction can take a different Compressor after construction, letting one AttendanceTimesFormatter instance run against NoneCompressor and GzipCompressor back to back. Running two data sources against two compressors produced four output combinations from just six classes total, well short of what straight inheritance would need once a third dimension gets added.

Gamma citation

Gamma et al. (1994, p. 151) describes Bridge as a way to "decouple an abstraction from its implementation so that the two can vary independently." Here, the formatter varies independently from where the data comes from and how the output gets compressed. Changing the compression strategy never touches TimesFormatter, and adding a new data source never touches the formatter subclasses.